

Lab 3: Architectural Pattern Justification

Varshini A | PES1UG24CS517 | Problem Statement #28: Inter-City Freight Load Matching Marketplace

Architecture Selection: We chose Microservices Architecture for the Inter-City Freight Load Matching Marketplace System.

Architectural Choice: Microservices. The marketplace is split into independently deployable services (Identity & Verification, Load Posting, Bidding & Matching, Proof-of-Delivery, Payment & Escrow, Notification) behind an API Gateway. Each service owns its own data (the diagram shows the ledger database for Payment & Escrow) and communicates only through the provided/required interfaces in the component diagram. Scoring the three styles against what this system needs:

Criterion (this scenario)	Layered	Client-Server	Microservices (selected)
Scaling for auction bid spikes	Poor: whole stack scales as one unit	Poor: one central server is the bottleneck	Good: scale only Bidding & Matching
Fault isolation (payments must survive other failures)	Poor: a fault in one layer can stall all	Poor: single point of failure	Good: failures stay inside one service
Security isolation of payment data	Fair: shared codebase and database	Fair: one trust zone on one server	Good: own service, ledger DB and credentials
Data consistency	Good: single database	Good: central database	Fair: distributed; needs idempotent calls
Simplicity to build and operate	Good: familiar and simple	Good: simplest to deploy	Poor: more services and DevOps effort

Verdict: Microservices wins on the three criteria this marketplace depends on most (scaling, fault isolation, payment security). Its costs, operational effort and distributed consistency, are accepted and managed with an API Gateway, asynchronous notification calls and idempotent payment calls.

Reason 1: Bursty, uneven load. Auction windows cause sharp spikes in bid traffic on popular lanes, while payment and proof-of-delivery volumes stay low. Only the Bidding & Matching Service needs to be scaled out during a rush, instead of replicating the whole application (Layered) or overloading one central server (Client-Server).

Reason 2: Fault isolation and independent change. Carrier permit rules, bidding logic and the bank/escrow integration change at different rates and can be deployed separately. A failure in load search or notifications must never block milestone payment after proof-of-delivery sign-off, and a failure inside one service stays contained instead of taking down the marketplace.

Security Advantage: Payment isolation with least privilege. Only the Payment & Escrow Service holds the bank API credentials and its own ledger database, and it is reachable only through its three published interfaces (IEscrowFunding, IMilestoneRelease and IPaymentStatus, the last one behind the gateway's JWT check). Calls between services are also authenticated (for example with mutual TLS), so no other service can reach it without valid credentials. A compromise of a less critical service such as Load Posting therefore cannot expose funds or financial records.

Performance Benefit: A short critical path for payments. Notifications are sent as non-blocking asynchronous calls, so the path from proof-of-delivery submission to fund release is only Proof-of-Delivery → Payment & Escrow → bank API. Bid surges and slow notifications cannot delay it. This is how the design is meant to meet the problem statement's 60-second escrow-release target (to be confirmed with a load test).